Programming Assignment 3 (PA3)

Magical Demon Hunters Game
User-Defined Types, Encapsulation, Operator Overloading, Interacting Objects
Out: March 11, 2026 -- DUE: April 7, 2026
EC327 Introduction to Software Engineering – Fall 2026

Total: 200 points

- *You may use any development environment you wish, as long as it is ANSI C++ 11 compatible. Please make sure your code compiles and runs properly under the Linux/Unix environment on the PHO 305/307 (or eng-grid) machines before submitting.*
- *PAs may be submitted up to a week late at the cost of a **30% fixed penalty** (e.g., submitting a day late and a week late is equivalent). It is in your best interest to complete as many checkpoints as possible before the deadline. If you have missing implementations in your original submission, you may complete and submit the missing solutions during the following week. Any submissions after the deadline will be subject to the 30% penalty. No credit will be given to solutions submitted after the 1-week late submission period following the deadline.*
- *Follow the assignment submission guidelines in this document or you will lose points.*

Submission Format (Must Read)

- Use the **exact** file names specified in each problem for your solutions.
- Push your changes to the repository generated by GitHub Classroom.
- Complete submissions should have **22 files.**
- Please do **NOT** submit *.exe and *.o or any other files that are not required by the problem.
- **Code must compile in order to be graded. Otherwise, this is an automatic zero.**
- Comment your code (good practice!). We **may** use your comments when grading.

Coding Style (reminder from PA2) As you become an experienced programmer, you will start to realize the importance of good programming style. There are many coding "guidelines" out there and it is important that you begin to adopt your own. Naming conventions will help you recognize variable names, functions, constants, classes etc. Similarly, there are many ways to elegantly format your code so that it is easy to read. All of these issues do not affect compilation and hence are easy to overlook at this stage in your development as a programmer. However, your ability (or inability) to create clean, readable code could be the difference in your career when you leave college.

Good reference: https://google.github.io/styleguide/cppguide.html If you find any other good references, feel free to post the link on Piazza!

Class Specifications

Each class and its members are described below by listing its name, the prototypes for the member functions, and the names and types of the member variables. You **MUST** use the prototypes, types, and names in your program as specified here, in the same upper/lower case. Function argument names are allowed to be different. **Failure to follow these specifications will result in lost points.**

Checkpoint 2

Building (10 points)

This class is the base class for all building type objects in the game. It inherits from GameObject and is responsible for tracking the total number of Mages that enter or leave a building.

Private Members

- `unsigned int mage_count`
  - The number of Mages currently within this Building
  - Initial value should be set to 0.

Constructors

- The default constructor that initializes the member variables to their initial values:
  - `display_code` should be 'B'
  - Should print out the message **"Building default constructed"**.
- `Building(char in_code, int in_Id, Point2D in_loc)`
  - Initializes the id number to `in_id`, and the location to `in_loc`, `display_code` to `in_code`, and remainder of the member variables to their default initial values.
  - Prints out the message **"Building constructed"**.

Public Member Functions

- `void AddOneMage();`
  - Increments `mage_count` by one.
- `void RemoveOneMage();`

  - Decrements `mage_count` by one.

- `void ShowStatus();`
  - Prints **"(display_code)(id) located at (location)"**
  - Prints "**(mage_count) mage(s) is/are in this building"**
  - Note: Instead of is/are try to conditionally select which of the verbs to use when printing

- `bool ShouldBeVisible();`
  - Returns true because buildings are always visible

ManaSpire

(20 points)

This class has a location and a set amount of magic crystals. It also has a `display_code` letter and an id number that are used to help identify the object in the output. `ManaSpire` inherits from `Building`.

State Machine Enums:

The following enumerated type should be declared in the `ManaSpire.h`

```
enum ManaSpireStates {
    CRYSTALS_AVAILABLE = 0,
    NO_CRYSTALS_AVAILABLE = 1
};
```

Private Members

- `unsigned int crystal_capacity`
  - The maximum number of crystals this ManaSpire can hold.
- `unsigned int num_crystals_remaining`
  - The amount of crystals currently in this ManaSpire
  - Initial value should be set to `crystal_capacity`.
- `double cost_per_crystal`
  - The per crystal cost in the ManaSpire

Constructors

- The default constructor that initializes the member variables to their initial values:
  - `display_code` should be 'S'
  - `crystal_capacity` should be 100 **(Piazza updates may change this as we tweak the game)**
  - `num_crystals_remaining` should be set to `crystal_capacity`
  - `cost_per_crystal` should be set to 5 **(Piazza updates may change this as we tweak the game)**
  - `state` should be `CRYSTALS_AVAILABLE`.
  - Should print out the message **"ManaSpire default constructed"**.

- `ManaSpire (int in_id, double crystal_cost, unsigned int crystal_cap, Point2D in_loc);`
  - Initializes the id number to `in_id`, and the location to `in_loc`, `cost_per_crystal` to `crystal_cost` and `crystal_capacity` to `crystal_cap`. The rest of the variables are assigned default values.
  - Prints out the message **"ManaSpire constructed"**.
  - `state` should be `CRYSTALS_AVAILABLE`.

Public Member Functions

- `bool HasCrystals()`
  - Returns `true` if this ManaSpire contains at least one crystal.
  - Returns `false` otherwise.
- `unsigned int GetNumCrystalsRemaining()`
  - Returns the number of crystals remaining in this ManaSpire.
- `bool CanAffordCrystal(unsigned int crystal, double budget)`
  - Returns true if this Mage can afford to purchase crystal with the given budget.
- `double GetCrystalCost(unsigned int crystal)`
  - Returns the cost (in gold pieces) for the specified number of crystals
- `unsigned int DistributeCrystals(unsigned int crystals_needed)`
  - If the amount `num_crystals_remaining` in the ManaSpire is greater than or equal to `crystals_needed`, it subtracts `crystals_needed` from ManaSpire amount and returns `crystal_needed`. If the amount of crystals in the ManaSpire is less, it returns the ManaSpire current amount, and the ManaSpire crystal amount is set to 0.
- `bool Update()`
  - If the ManaSpire has no crystals remaining
    * Its `state` is set to `NO_CRYSTALS_AVAILABLE`
    * `display_code` is changed to 'c'
    * Prints the message **"ManaSpire (id number) has ran out of crystals."**
    * Returns `true` if crystal is depleted; returns `false` if it is not depleted.
  - This function shouldn't keep returning true if the ManaSpire has no crystal remaining. It should return true ONLY at the time when the ManaSpire runs out of crystal, and return false for later `Update()` function calls.

- `void ShowStatus()`
  - Prints out the status of the object:
    * **“ManaSpire Status: “**
    * Calls `Building::ShowStatus()`
    * **“Gold pieces per crystal: (cost_per_crystal)”**
    * **“has (crystal_remaining) crystal(s) remaining. “**

DemonHideout

(20 points)
A DemonHideout object has a location and an amount of battles. You can battle with your Mage in a DemonHideout and earn experience. It also has a `display_code` letter and id number that are used to help identify the object in the output. It has the following members. For clarity, the private members are described first, although normally they are declared after the public members in the code. DemonHideout objects should inherit from Building.

State Machine Enums:

The following enumerated type should be declared in the DemonHideout.h

```
enum DemonHideoutStates{
    NOT_DEFEATED = 0,
    DEFEATED = 1
};
```

Private Members

- `unsigned int num_battle_remaining`
  - The amount of battles remaining in the DemonHideout.
- `unsigned int max_number_of_battles`
  - Number of battles for this DemonHideout
- `unsigned int mana_cost_per_battle`
  - Mana lost for a single hideout battle (it depletes your mana when you battle)
- `double gold_cost_per_battle`
  - Entry cost (in gold) for single battle
- `unsigned int experience_per_battle`
  - Amount of experience gained from each class.

Constructors

- The default constructor that initializes the member variables to their initial values:
  - Display Code: ‘D’

- State: `NOT_DEFEATED`.
- `max_number_of_battles` should be 10 **(Piazza updates may change this as we tweak the game)**
- `num_battle_remaining` should be set to `max_number_of_battles`
- `mana_cost_per_battle` should be 1 **(Piazza updates may change this as we tweak the game)**
- `gp_cost_per_battle` should be 1.0 **(Piazza updates may change this as we tweak the game)**
- `experience_per_battle` should be 2 **(Piazza updates may change this as we tweak the game)**
- Prints out the message **"DemonHideout default constructed"**.

- `DemonHideout(unsigned int max_battle, unsigned int mana_loss, double cost, unsigned int exp_per_battle int in_id, Point2D in_loc)`
  - Initializes the id number to `in_id`
  - `max_number_of_battles` to `max_battle`,
  - `mana_cost_per_battle` to `mana_loss`,
  - `experience_per_battle` to `exp_per_battle`,
  - `gold_cost_per_battle` to cost and location to `in_loc`
  - Initializes the rest of the member variables to default values
  - Prints out the message **"DemonHideout constructed"**.

Public Member Functions

- `double GetGoldCost(unsigned int battle_qty)`
  - Returns the cost of battling `battle_qty` times.
- `unsigned int GetManaCost(unsigned int battle_qty)`
  - Returns the amount of mana points required for `battle_qty` battles
- `unsigned int GetNumBattlesRemaining()`
  - Returns the number of battles remaining in this DemonHideout.
- `bool IsAbleToBattle(unsigned int battle_qty, double budget, unsigned int mana)`

- Returns true if a Mage in a DemonHideout with a given budget and mana can request to take `battle_qty` battle
- Returns false otherwise

- `unsigned int DemonBattle(unsigned int battle_units)`
  - Subtracts battles from `num_battles_remaining` if this DemonHideout has enough units. If the amount of battles requested is greater than the amount available at this DemonHideout, then `num_battles_remaining` will be used instead of `battle_units` when calculating experience gain.
  - Returns the amount of experience gained by winning the battles.
  - experience points can be calculated using (number of battles) * `experience_per_battle`
- `unsigned int GetExperiencePerBattle()`
  - Returns the experience gained from each battle.
- `bool Update()`
  - If the DemonHideout has zero battles remaining, set the state to `DEFEATED` and `display_code` to 'g'. Then print the message "**(display_code)(id) has been beaten"**.
  - Returns false if battles still remain within the DemonHideout.
  - This function shouldn't keep returning true if the DemonHideout is passed. It should return true ONLY at the time when the DemonHideout is defeated, and return false for later "`Update()`" function calls.
- `bool passed ();`
  - Returns true if battles remaining is 0
- `void ShowStatus()`
  - Prints out the status of the object by calling GameObject's show status and then the values of its member variables:
    * **"DemonHideoutStatus: "**
    * Calls `Building::ShowStatus()`
    * **"Max number of battles: (max_number_of_battles)"**
    * **"Mana cost per battle: (mana_cost_per_battle)"**
    * **"Gold per battle: (gold_cost_per_battle)"**
    * **"Experience per battle: (experience_per_battle)"**
    * **"(num_battles_remaining) battle(s) are remaining for this DemonHideout"**

How the `Building`, `ManaSpire` and `DemonHideout` Objects Behave

These objects also change their state, but they do so simply. The `ManaSpire::Update()` function simply checks to see if there are any remaining crystals for distribution. If there are no more crystals remaining then its state changes from `CRYSTALS_AVAILABLE` to `NO_CRYSTALS_AVAILABLE` and the display code changes from 'S' to 's'; true is returned to signify this event occurred. Similarly, the `DemonHideout::Update()` function checks to see if the amount of battles remaining is equal to zero; if it is then the `DemonHideout`'s state changes from `NOT_DEFEATED` to `DEFEATED` and the display code is changed from 'D' to 'd'. They both inherit from `GameObject`. When a Mage enters a building, the Building's `mage_count` increases by one. Likewise, when a Mage leaves a building, its `mage_count` decreases by one.

The constructors should output the appropriate messages such as "Building constructed", "ManaSpire default constructed" and "DemonHideout default constructed" as before. The `ShowStatus()` functions for each building should output "**(class name) status:**", then call the shadowed `Building::ShowStatus()` function, and then output the information specific to that building. Later, when the `Mage` objects are created, you should see the proper sequence of messages from the constructors.

CHECKPOINT II

Write the `Building`, `DemonHideout` and `ManaSpire` classes. To test these classes, also write a `TestCheckpoint2.cpp`. Instantiate multiple objects of these classes in the main function and test out their functions (e.g. `Update()`, `DistributeCrystal()`, `AddOneMage()`, etc.) in order to ensure their proper behavior. For example, call each object's `ShowStatus()` method after calling their `Update()` method.

DO NOT MOVE ON UNTIL YOU ARE 100% SURE YOU UNDERSTAND THIS CHECKPOINT

If you do not make sure that `Building`, `ManaSpire` and `DemonHideout` work fully, the rest of your game will NOT work. Use this time to start to modify the provided makefile to also work for testing Checkpoint 2. DON'T RUSH THROUGH THIS.